Week 2 - AWS Lambda versions, aliases, weighted aliases and SAM DeploymentPreference

DOP-C02 Domain 1 (SDLC Automation). Exam focus: immutable versions vs $LATEST, aliases as stable pointers, weighted alias canaries, and safe automated Lambda deployments with SAM and CodeDeploy (canary or linear, alarms, hooks, automatic rollback).

Part 1 - Versions

$LATEST and published versions

	$LATEST	Published version (1, 2, 3 ...)
Editable	**Yes**: every code or config change updates it	**No**, it is **immutable** (a snapshot)
Created by	Always exists	`PublishVersion` API, `aws lambda publish-version`, `update-function-code --publish`, or SAM `AutoPublishAlias`
Numbering	n/a	Increases by one and **is never reused**, even after a version is deleted
Provisioned concurrency	**Not allowed**	Allowed
Weighted alias	**Not allowed**	Allowed
SnapStart	Not applied	Applied when the version is published (Java, Python, .NET)
Use for	Development	Testing, staging, production

What a version freezes

A version is a snapshot of **both the code and the configuration**:

- code (the zip or container image digest),
- runtime, handler and architecture,
- memory, timeout and ephemeral storage,
- **environment variables**,
- layers (by layer version), VPC settings and the execution role,
- dead-letter queue and on-failure destination settings.
- **Exam trap:** environment variables are frozen in the version. To change a setting in production you must **publish a new version** and repoint the alias. Settings that change per environment, or must change without a deployment, belong in **SSM Parameter Store or Secrets Manager**, read at runtime.
- **Not part of a version: reserved concurrency** is set per function and shared by all versions and aliases. Tags also apply to the whole function.
- **Provisioned concurrency** is the opposite: it is set per **version or alias**.

Publishing rules

- If nothing has changed since the last published version, `PublishVersion` **returns the existing version** instead of creating a new one.
- Pass `CodeSha256` to `PublishVersion` so it only publishes if the code is what you expect. This guards against someone changing `$LATEST` in between.
- Old versions can be deleted. You **can't delete a version that an alias points to**.
- Every version's code counts toward the **Lambda code storage quota** (75 GB per Region by default). Clean up old versions, for example with a pipeline step that keeps the last N.

ARNs: qualified vs unqualified

ARN	Example	Invokes
Unqualified	`arn:aws:lambda:us-east-1:111122223333:function:orders`	`$LATEST`
Qualified (version)	`...:function:orders:7`	Version 7
Qualified (alias)	`...:function:orders:prod`	Whatever version `prod` points to

- With an unqualified ARN you can't use provisioned concurrency or weighted routing, and callers run whatever was last saved. **Production callers should use an alias ARN.**
- **Resource-based policies** (permissions for other services to invoke) are set **per qualifier**. A permission on the unqualified function doesn't cover `:prod`, and the reverse is also true.

Monitoring by version

- CloudWatch dimensions: `FunctionName`, `Resource` (function plus alias or version), and `ExecutedVersion` (the version that actually ran). Use `ExecutedVersion` to compare old and new during a weighted rollout.
- The log stream name includes the version, for example `2026/10/28/[7]abc...`.
- The synchronous `Invoke` response includes the **executed version**.

Part 2 - Aliases

An alias is a named, movable pointer to a version, such as `dev`, `beta` or `prod`.

- It has its own ARN: `...:function:orders:prod`.
- It can point to a **published version**, or to `$LATEST` (but then no weights and no provisioned concurrency).
- It **can't point to another alias**.
- Each alias can have its own resource policy, provisioned concurrency, event source mappings, function URL and destinations.

Why use aliases

1. **Stable endpoint:** callers (API Gateway, S3, SNS, EventBridge, SQS event source mappings, other services) use `orders:prod` and never change.
2. **Promotion:** test version 8 on `beta`, then point `prod` at 8.
3. **Instant rollback:** point `prod` back at version 7. No redeploy, no code change.
4. **Environments in one function:** `dev` to `$LATEST`, `test` to 8, `prod` to 7. For strict isolation, separate functions or accounts per environment are still preferred.

```
aws lambda publish-version --function-name orders --description "release 1.4"
aws lambda create-alias --function-name orders --name prod --function-version 7
aws lambda update-alias --function-name orders --name prod --function-version 8   # promote
aws lambda update-alias --function-name orders --name prod --function-version 7   # roll back
```

Integrations with aliases

- **API Gateway:** the integration URI can include the alias, or use a **stage variable**: `arn:...:function:orders:${stageVariables.lambdaAlias}`, so the `dev` stage uses `dev` and the `prod` stage uses `prod`.
  - Grant API Gateway `lambda:InvokeFunction` **on each alias** (`aws lambda add-permission --qualifier prod`). If you forget, API Gateway returns **500 Internal server error** with an "Invalid permissions on Lambda function" log.
- **Event source mappings** (SQS, Kinesis, DynamoDB Streams, MSK): point the mapping at the **alias ARN**. If it points at the unqualified function, production processes untested `$LATEST` code.
- **Push sources** (S3, SNS, EventBridge): set the target to the alias ARN, and add a permission for that alias.
- **Function URLs** can be attached to an alias.
- **Provisioned concurrency** on the `prod` alias keeps warm instances for the version it points to. **Application Auto Scaling** can scale it on a schedule or on utilization.

Part 3 - Weighted aliases (alias routing configuration)

An alias can send traffic to **two versions**: its main version plus **one additional version** with a weight.

```
# prod: 90% to version 7, 10% to version 8
aws lambda update-alias --function-name orders --name prod \
  --function-version 7 \
  --routing-config 'AdditionalVersionWeights={"8"=0.1}'

# finish: everything on 8, remove the routing config
aws lambda update-alias --function-name orders --name prod \
  --function-version 8 --routing-config 'AdditionalVersionWeights={}'
```

Rules

- **Two versions** at most (main plus one additional).
- **Both must be published versions.** `$LATEST` is not allowed.
- Both versions must use the **same execution role**.
- Both must have the **same dead-letter queue configuration**, or both none.
- The additional version's weight is 0.0 to 1.0. The main version gets the rest.
- Routing is **random per invocation**, not sticky per user. If the versions are incompatible (different response formats or data schemas), use blue/green or a feature flag instead.
- Watch the `ExecutedVersion` dimension to compare errors and duration for the two versions.
- **Provisioned concurrency** on a weighted alias is split between the versions by weight.

Manual vs automated shifting

	Manual weighted alias	CodeDeploy (SAM, CloudFormation or pipeline)
Who changes weights	You or a script	**CodeDeploy**, on a schedule (canary or linear)
Health checks	You watch metrics	**CloudWatch alarms** checked automatically
Rollback	You repoint the alias	**Automatic** on an alarm or a failed hook
Tests before and after the shift	Your own	**PreTraffic and PostTraffic hooks**

- **Exam answer:** gradual shift with **automatic rollback** and **minimal custom code** means **SAM** `AutoPublishAlias` **+** `DeploymentPreference` (CodeDeploy), not a hand-written Lambda that updates weights.

Part 4 - SAM AutoPublishAlias and DeploymentPreference

```
Transform: AWS::Serverless-2016-10-31

Resources:
  OrdersFunction:
    Type: AWS::Serverless::Function
    Properties:
      Handler: app.handler
      Runtime: python3.12
      CodeUri: src/
      AutoPublishAlias: live
      ProvisionedConcurrencyConfig:
        ProvisionedConcurrentExecutions: 5      # set on the alias
      DeploymentPreference:
        Type: Canary10Percent5Minutes
        Alarms:
          - !Ref OrdersErrorsAlarm
          - !Ref OrdersLatencyAlarm
        Hooks:
          PreTraffic: !Ref PreTrafficHook
          PostTraffic: !Ref PostTrafficHook
        TriggerConfigurations:                  # optional SNS notifications
          - TriggerEvents: [DeploymentFailure, DeploymentRollback]
            TriggerName: orders-deploy-alerts
            TriggerTargetArn: !Ref AlertsTopic
        # Role: !GetAtt MyCodeDeployRole.Arn    # optional custom service role
        # Enabled: true                         # false skips gradual shifting

  PreTrafficHook:
    Type: AWS::Serverless::Function
    Properties:
      FunctionName: CodeDeployHook_orders_PreTraffic   # prefix matters
      Handler: hooks.pre
      Runtime: python3.12
      CodeUri: hooks/
      DeploymentPreference:
        Enabled: false
      Environment:
        Variables:
          TARGET: !Ref OrdersFunction.Version          # the new version to test
      Policies:
        - Statement:
            - Effect: Allow
              Action: codedeploy:PutLifecycleEventHookExecutionStatus
              Resource: "*"
            - Effect: Allow
              Action: lambda:InvokeFunction
              Resource: !Ref OrdersFunction.Version
```

What AutoPublishAlias does

- On every deployment where the **code changes**, SAM **publishes a new version** and creates or updates the alias (here `live`) to point to it.
- SAM detects code changes from the code hash. `AutoPublishCodeSha256` forces a new version when something outside `CodeUri` changes.
- It fixes the plain CloudFormation problem: `AWS::Lambda::Version` **doesn't publish again** on a code change unless its logical ID changes.
- It exposes `!Ref Function.Version` and `!Ref Function.Alias`.
- `DeploymentPreference` **requires** `AutoPublishAlias`, because CodeDeploy shifts the alias between versions.

What DeploymentPreference does

- SAM generates a CodeDeploy **application** (`ServerlessDeploymentApplication`), a **deployment group** per function, a **service role** with the `AWSCodeDeployRoleForLambda` managed policy (unless you give `Role`), and an `UpdatePolicy: CodeDeployLambdaAliasUpdate` on the alias.

- When the stack updates, **CloudFormation hands the alias change to CodeDeploy** and **waits**. If CodeDeploy rolls back, the stack update **fails and rolls back** too.

Deployment types

Type	Behaviour
`Canary10Percent5Minutes`	10% for 5 minutes, then 100%
`Canary10Percent10Minutes`	10% for 10 minutes, then 100%
`Canary10Percent15Minutes`	10% for 15 minutes, then 100%
`Canary10Percent30Minutes`	10% for 30 minutes, then 100%
`Linear10PercentEvery1Minute`	+10% every minute (10 steps)
`Linear10PercentEvery2Minutes`	+10% every 2 minutes
`Linear10PercentEvery3Minutes`	+10% every 3 minutes
`Linear10PercentEvery10Minutes`	+10% every 10 minutes
`AllAtOnce`	100% immediately (hooks and alarms still run)

- These map to CodeDeploy configurations such as `CodeDeployDefault.LambdaCanary10Percent5Minutes`.
- A **custom CodeDeploy deployment configuration** (for example canary 20% for 15 minutes) can be referenced by name in `Type`.
- Canary = **two steps**. Linear = **equal steps**. AllAtOnce = one step.

Deployment flow

1. SAM publishes the new version (v8). The alias still points to v7.
2. **PreTraffic hook** runs (CodeDeploy `BeforeAllowTraffic`). It tests v8 directly before any user traffic.
3. CodeDeploy shifts the alias weights step by step (for example 10%, wait 5 minutes, then 100%), checking the **alarms** the whole time.
4. **PostTraffic hook** runs (`AfterAllowTraffic`) and checks v8 with all the traffic.
5. The deployment succeeds; the alias points 100% to v8.

- **Rollback:** an **alarm in ALARM** during the deployment, or a **hook that reports Failed or times out**, makes CodeDeploy point the alias **back to v7 at 100%**, and the CloudFormation update fails.

Hooks

- A hook is a Lambda function. CodeDeploy calls it with a `DeploymentId` and a `LifecycleEventHookExecutionId`.
- The hook **must call** `codedeploy:PutLifecycleEventHookExecutionStatus` with `Succeeded` or `Failed`. If it never reports, it **times out after 1 hour** and the deployment fails.
- The default service role policy (`AWSCodeDeployRoleForLambda`) lets CodeDeploy **invoke hook functions whose names start with** `CodeDeployHook_`. Name hooks that way, or give a custom `Role`.
- The hook's own role needs `codedeploy:PutLifecycleEventHookExecutionStatus` and `lambda:InvokeFunction` on the version it tests.
- Set `DeploymentPreference: Enabled: false` (or leave it out) on the hook functions themselves.

```
import boto3, os, json
cd = boto3.client("codedeploy"); lam = boto3.client("lambda")

def pre(event, context):
    status = "Succeeded"
    try:
        r = lam.invoke(FunctionName=os.environ["TARGET"], Payload=json.dumps({"test": True}))
        if r.get("FunctionError"): status = "Failed"
    except Exception:
        status = "Failed"
    cd.put_lifecycle_event_hook_execution_status(
        deploymentId=event["DeploymentId"],
        lifecycleEventHookExecutionId=event["LifecycleEventHookExecutionId"],
        status=status)
```

Alarms to use

- `Errors` and `Throttles` (sum), `Duration` p99, custom business metrics, API Gateway `5XXError` and `Latency`.
- Scope alarms to the **new version** with the `ExecutedVersion` dimension (or to the alias with `Resource`), so old-version errors don't trigger a rollback or hide a bad new version.

Part 5 - The same thing without SAM

CloudFormation

```
OrdersVersion:
  Type: AWS::Lambda::Version            # change the logical ID to publish again
  Properties:
    FunctionName: !Ref OrdersFunction

OrdersAlias:
  Type: AWS::Lambda::Alias
  Properties:
    FunctionName: !Ref OrdersFunction
    FunctionVersion: !GetAtt OrdersVersion.Version
    Name: live
    # RoutingConfig:                    # manual weights
    #   AdditionalVersionWeights:
    #     - FunctionVersion: "8"
    #       FunctionWeight: 0.1
  UpdatePolicy:
    CodeDeployLambdaAliasUpdate:
      ApplicationName: !Ref CodeDeployApp
      DeploymentGroupName: !Ref OrdersDeploymentGroup
      BeforeAllowTrafficHook: !Ref PreTrafficHook
      AfterAllowTrafficHook: !Ref PostTrafficHook
```

CodeDeploy directly (for example from CodePipeline)

```
version: 0.0
Resources:
  - orders:
      Type: AWS::Lambda::Function
      Properties:
        Name: orders
        Alias: live
        CurrentVersion: "7"
        TargetVersion: "8"
Hooks:
  - BeforeAllowTraffic: CodeDeployHook_orders_PreTraffic
  - AfterAllowTraffic: CodeDeployHook_orders_PostTraffic
```

- Lambda deployments have **only two hooks**: `BeforeAllowTraffic` and `AfterAllowTraffic`.
- CodeDeploy for Lambda is always **traffic shifting on an alias**. There is no in-place option.

CDK

- `LambdaDeploymentGroup` with `alias`, `deploymentConfig: LambdaDeploymentConfig.CANARY_10PERCENT_5MINUTES`, `alarms`, and `preHook` / `postHook`.

Part 6 - Troubleshooting

Symptom	Cause and fix
API Gateway returns 500 Invalid permissions after switching to an alias	Add a resource policy for **that alias** (`add-permission --qualifier`)
Production runs untested code	The trigger or event source mapping uses the unqualified ARN ($LATEST). Point it at the **alias**
Can't add weights to an alias	A version is $LATEST, the versions have different roles or DLQ settings, or more than two versions
Changed an environment variable but production is unchanged	Versions are immutable. **Publish a new version** and update the alias, or use Parameter Store
CloudFormation doesn't create a new version on a code change	`AWS::Lambda::Version` needs a new logical ID. Use SAM **AutoPublishAlias**
SAM deployment hangs about an hour, then rolls back	The hook never called `PutLifecycleEventHookExecutionStatus`
Hook fails with AccessDenied	Name doesn't start with `CodeDeployHook_` under the default role, or the hook's role lacks permissions
Rolled back although the new version looked fine	The alarm included old-version errors. Scope it with `ExecutedVersion`
Code storage limit reached	Too many old versions. Delete old versions automatically
Users get mixed responses during a rollout	Weighted routing is per request, not sticky. Make the versions backward-compatible
Cold starts after a release	**Provisioned concurrency on the alias**, or SnapStart on published versions
Need the fastest manual rollback	`update-alias` back to the previous version (seconds, no redeploy)

Quick memory summary

- **$LATEST** is mutable. **Versions** are immutable snapshots of code and config (including env vars), numbered and never reused. A version an alias points to can't be deleted.
- **Unqualified ARN** = $LATEST. **Qualified** = `:7` or `:prod`. Permissions are **per qualifier**.
- **Alias** = a movable pointer to a version, a stable ARN for callers. Promote or roll back by repointing. It carries provisioned concurrency and its own permissions.
- **Weighted alias:** 2 published versions only, same role, same DLQ, random per request. Watch **ExecutedVersion**.
- **SAM:** `AutoPublishAlias` publishes a version and updates the alias. `DeploymentPreference` sets the Canary, Linear or AllAtOnce `Type`, **Alarms**, and **PreTraffic / PostTraffic** hooks, run by CodeDeploy. An alarm or failed hook gives an **automatic rollback** and the stack update fails.
- **Hooks** must call `PutLifecycleEventHookExecutionStatus`, have a 1-hour limit, and need the `CodeDeployHook_` prefix with the default role.
- **Without SAM:** `AWS::Lambda::Alias` with `UpdatePolicy: CodeDeployLambdaAliasUpdate`, or a CodeDeploy Lambda appspec (`CurrentVersion`, `TargetVersion`, `BeforeAllowTraffic`, `AfterAllowTraffic`).